

DataVault Backend API Reference

backup-service & client-service — REST API documentation

Endpoints: 8 (backup-service) + 104 (client-service) = 112 total

Generated 2026-08-25

backup-service API Reference

Base URL: `/api` (health check) and `/api/v1/*` (all other routes).

Auth: Every route under `/api/v1/*` is protected by the `internalAuth` middleware — it requires header `X-Internal-Secret: <INTERNAL_SECRET>` sent by client-service (the only caller of this internal service). A missing/incorrect secret returns `401 unauthorized`. `GET /api/health` is public.

Response envelope (all endpoints): `{ success: boolean, message: string, data: any, meta: object }`

Health

GET /api/health

Liveness check.

Auth: None (public).

Success response — 200

```
{
  "success": true,
  "message": "Service is running",
  "data": null,
  "meta": {}
}
```

Backup Job

Base path: `/api/v1/backup-job`

POST `/api/v1/backup-job`

Creates a new backup job record and kicks off the backup run in the background (fire-and-forget); responds immediately with the created job.

Auth: `X-Internal-Secret` header required.

Request body

```
{
  "userId": "usr_8f3c1a2e",
  "backupConfigId": "cfg_7d21f9b4",
  "source": {
    "access_token": "00D5j000000ABCD!AQEAQP...",
    "refresh_token": "5Aep861aQ9cS9CjXz...",
    "instanceUrl": "https://mycompany.my.salesforce.com",
    "crmName": "salesforce",
    "crmId": "00D5j000000ABCDEAU",
    "object": [
      {
        "id": "obj_001",
        "name": "Account",
        "condition": { "type": "AND" },
        "field": [
          { "name": "Industry", "filter": { "value": "Technology", "operator": "=" } }
        ],
        "parentObjects": [{ "id": "obj_000", "name": "Organization" }]
      }
    ]
  },
  "destination": {
    "type": "S3",
    "config": {
      "bucketName": "acme-datavault-backups",
      "region": "ap-south-1",
      "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
      "secretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
      "folderPath": "backups/salesforce/account"
    }
  },
  "lastUpdatedAt": "2026-08-20T10:15:00.000Z",
  "spaceId": "spc_4471"
}
```

Success response — 201

```
{
  "success": true,
  "message": "Record created successfully",
  "data": {
    "backupJobId": "bkj_9a12ef34",
    "status": "PENDING"
  },
  "meta": {}
}
```

Error response — 400 (validation failure, e.g. missing `source.access_token`)

```
{
  "success": false,
  "message": "\"source.access_token\" is required",
  "data": null,
  "meta": {}
}
```

GET /api/v1/backup-job/resume

Resumes a previously interrupted (e.g. server-restarted) backup job by id and re-runs it in the background.

Auth: X-Internal-Secret header required.

Query params

Name	Type	Required	Description
id	string	yes	<code>backupJobId</code> to resume

Success response — 200

```
{
  "success": true,
  "message": "Backup job resumed successfully",
  "data": {
    "backupJobId": "bkj_9a12ef34",
    "status": "RUNNING"
  },
  "meta": {}
}
```

Error response — 400 (job not found)

```
{
  "success": false,
  "message": "Record not exists",
  "data": null,
  "meta": {}
}
```

POST /api/v1/backup-job/archival

Creates a new archival job (Hudi-only, no Delta table) and runs it in the background. Same request/response shape as `POST /backup-job` but validated with the archival schema (richer object/field/condition/SOQL support, no top-level `object` filter shorthand).

Auth: X-Internal-Secret header required.

Request body

```
{
  "userId": "usr_8f3c1a2e",
  "backupConfigId": "cfg_arc_5512",
  "source": {
    "access_token": "00D5j000000ABCD!AQEAQP...",
    "refresh_token": "5Aep861aQ9cS9CjXz...",
    "instanceUrl": "https://mycompany.my.salesforce.com",
    "crmName": "salesforce",
    "crmId": "00D5j000000ABCDEAU",
    "object": [
      {
        "id": "obj_001",
        "name": "Case",
        "fieldApiName": "ClosedDate",
        "condition": { "type": "SOQL", "soqlQuery": "ClosedDate < LAST_N_YEARS:2" },
        "type": "OBJECT",
        "field": [
          {
            "name": "ClosedDate",
            "dataType": "DATE",
            "filter": { "operator": "<", "value": "2024-01-01" }
          }
        ],
        "children": []
      }
    ]
  },
  "destination": {
    "type": "S3",
    "config": {
      "bucketName": "acme-datavault-archives",
      "region": "ap-south-1",
      "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
      "secretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY",
      "folderPath": "archives/salesforce/case"
    }
  },
  "lastUpdatedAt": "2026-08-20T10:15:00.000Z",
  "spaceId": "spc_4471"
}
```

Success response — 201

```
{
  "success": true,
  "message": "Record created successfully",
  "data": {
    "backupJobId": "bkj_arc_77c1",
    "status": "PENDING"
  },
  "meta": {}
}
```

Error response — 400 (validation failure, e.g. `condition.type` is `SOQL` but `soqlQuery` missing)

```
{
  "success": false,
  "message": "\"source.object[0].condition.soqlQuery\" is required",
  "data": null,
  "meta": {}
}
```

GET /api/v1/backup-job/archival/resume

Resumes a previously interrupted archival job. Internally reuses the same resume path as a regular backup job resume, but re-runs it through the archival runner.

Auth: X-Internal-Secret header required.

Query params

Name	Type	Required	Description
id	string	yes	backupJobId (archival job) to resume

Success response — 200

```
{
  "success": true,
  "message": "Backup job resumed successfully",
  "data": {
    "backupJobId": "bkj_arc_77c1",
    "status": "RUNNING"
  },
  "meta": {}
}
```

Error response — 400 (job not found)

```
{
  "success": false,
  "message": "Record not exists",
  "data": null,
  "meta": {}
}
```

Glue

Base path: `/api/v1/glue`

POST `/api/v1/glue/ensure-compression-tables`

Called by the Spark compression job on completion. For each object, ensures the current-state Hudi table exists in the Glue Catalog (and the Delta table too, unless `isArchival` is true). Idempotent — existing tables are left untouched. Concurrent duplicate calls for the same `backupConfigId` are coalesced onto a single in-flight run.

Auth: `X-Internal-Secret` header required.

Request body

```
{
  "crmId": "00D5j000000ABCDEAU",
  "crmName": "salesforce",
  "backupConfigId": "cfg_7d21f9b4",
  "objectNames": ["Account", "Contact", "Opportunity"],
  "destConfig": {
    "bucketName": "acme-datavault-backups",
    "region": "ap-south-1",
    "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
    "secretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
    "folderPath": "backups/salesforce"
  },
  "isArchival": false
}
```

Success response — 200

```
{
  "success": true,
  "message": "Record created successfully",
  "data": {
    "ensured": ["Account", "Contact"],
    "failed": [
      { "objectName": "Opportunity", "error": "hudi: .hoodie metadata not found at s3://acme-datavault-backups/backups/sales"
    ]
  },
  "meta": {}
}
```

Error response — 400 (missing required params, e.g. empty `objectNames`)

```
{
  "success": false,
  "message": "Required parameters are missing",
  "data": null,
  "meta": {}
}
```

Realtime Backup

Base path: `/api/v1/realtime-backup`

POST `/api/v1/realtime-backup`

Entry point for every Salesforce realtime-change webhook hit, forwarded by client-service after it handles auth/config lookup/destination resolution. Upserts the backup job for the given `transactionId` + `objectApiName` + `operation` (deduplicating multiple HTTP hits belonging to the same Salesforce transaction), responds `202` immediately, then uploads the record(s) to S3 in the background.

Auth: `X-Internal-Secret` header required.

Request body

```
{
  "userId": "usr_8f3c1a2e",
  "backupConfigId": "cfg_7d21f9b4",
  "crmId": "00D5j000000ABCDEAU",
  "crmName": "salesforce",
  "destination": {
    "type": "S3",
    "config": {
      "bucketName": "acme-datavault-backups",
      "region": "ap-south-1",
      "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
      "secretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
      "folderPath": "realtime/salesforce/account"
    }
  },
  "realtimePayload": {
    "objectApiName": "Account",
    "operation": "UPDATE",
    "transactionId": "txn_5e6f7a8b9c",
    "recordId": "0015j000003ABCDEAU",
    "changedFields": { "Name": "Acme Corp", "Industry": "Technology" }
  },
  "spaceId": "spc_4471"
}
```

Success response — 202

```
{
  "success": true,
  "message": "Record created successfully",
  "data": {
    "backupJobId": "bkj_rt_2c9e11",
    "status": "RUNNING"
  },
  "meta": {}
}
```

Error response — 500 (unexpected failure, e.g. bad destination credentials surfaced before the 202 is sent)

```
{
  "success": false,
  "message": "internal_server_error",
  "data": null,
  "meta": {}
}
```

Restore Job

Base path: `/api/v1/restore`

POST `/api/v1/restore`

Triggers execution of a restore job that was previously created (in `PENDING` status) by client-service. Looks the job up by `restoreJobId` ; if found and still pending, responds `200` immediately and runs the restore in the background.

Auth: `X-Internal-Secret` header required.

Request body

```
{
  "restoreJobId": "rsj_3f8c21ab",
  "userId": "usr_8f3c1a2e",
  "source": {
    "backupConfigId": "cfg_7d21f9b4",
    "crmId": "00D5j000000ABCDEAU",
    "crmName": "salesforce",
    "bucketName": "acme-datavault-backups",
    "region": "ap-south-1",
    "encryptedKeys": { "accessKeyId": "enc:AQICAHj...", "secretAccessKey": "enc:AQICAHk..." },
    "folderPath": "backups/salesforce/account",
    "csvFilePath": "backups/salesforce/account/2026-08-20/part-0000.csv"
  },
  "destination": {
    "crmId": "00D5j000000WXYZEAU",
    "crmName": "salesforce",
    "encryptedTokens": { "access_token": "enc:AQICAHm...", "refresh_token": "enc:AQICAHn..." },
    "instanceUrl": "https://mycompany-sandbox.my.salesforce.com",
    "objects": [
      { "id": "obj_001", "name": "Account", "status": "PENDING", "processedRecordCount": 0, "failedRecordCount": 0 }
    ]
  },
  "conflict": {
    "restoreMode": "OVERWRITE",
    "edgeCases": {
      "onDuplicateRecord": "SKIP",
      "parentMissing": "SKIP"
    }
  },
  "mergeRule": {
    "default": "NEWEST_LAST_MODIFIED_DATE_WINS",
    "objects": [
      { "name": "Account", "fields": [{ "name": "Industry", "value": "SOURCE_ALWAYS_WINS" }] }
    ]
  }
},
"lastUpdatedAt": "2026-08-20T10:15:00.000Z"
}
```

Success response — 200

```
{
  "success": true,
  "message": "Record created successfully",
  "data": null,
  "meta": {}
}
```

Error response — 400 (job not found, or not in `PENDING` status)

```
{  
  "success": false,  
  "message": "Record not exists",  
  "data": null,  
  "meta": {}  
}
```


Client Service API — Part 1

Base URL: `/api/v1` (mounted at `/api` in `client-service/src/config/app/index.ts`, versioned router at `/v1`).

Response envelope (all endpoints, from `src/lib/response/index.ts`):

```
{ "success": true, "message": "fetch", "data": { }, "meta": { } }
```

`data` is `null` when no payload is returned. `meta` carries pagination info where applicable. HTTP status code is set per-endpoint by the controller.

Auth: session is a JWT in an `httpOnly` `accessToken` cookie (not a bearer header), set by `/auth/login`, `/auth/verify-otp` (login flow), or the social-login callback. Private routes additionally require `ac1Gateway`, which checks the caller's role permissions (`moduleKey.action` strings, e.g. `backup.read`) for modules that are not in an always-allowed list (`user`, `crm`, `crm-metadata`, `dashboard`, `destination`, `settings`).

Auth (/api/v1/auth) — public, no authenticate / aclGateway

POST /api/v1/auth/signup

Creates a user after their signup OTP has been verified. Rate-limited (authRateLimit : 10/15min/IP).

Request body

```
{
  "firstName": "Ava",
  "lastName": "Stone",
  "contact": { "email": "ava.stone@acme.com" },
  "password": "P@ssw0rd123",
  "authProvider": "email",
  "gender": "female"
}
```

Success response (201)

```
{ "success": true, "message": "User created successfully", "data": null, "meta": {} }
```

Error response (400 — OTP not verified/expired)

```
{ "success": false, "message": "OTP has expired", "data": null, "meta": {} }
```

POST /api/v1/auth/send-otp

Sends (persists, no real dispatch wired up yet) a signup/login/forgot-password OTP. Rate-limited (otpRateLimit : 5/15min/IP).

Request body

```
{ "otpType": "SIGNUP", "channel": "EMAIL", "contact": "ava.stone@acme.com" }
```

For channel: "MOBILE", contact is { "number": "9876543210", "dialCode": "+91" }.

Success response (200)

```
{ "success": true, "message": "OTP sent successfully", "data": null, "meta": {} }
```

Error response (400)

```
{ "success": false, "message": "Email is already registered", "data": null, "meta": {} }
```

POST /api/v1/auth/verify-otp

Verifies an OTP; if otpType is LOGIN, also logs the user in and sets accessToken / refreshToken cookies. Rate-limited (authRateLimit).

Request body

```
{ "otpType": "LOGIN", "channel": "EMAIL", "contact": "ava.stone@acme.com", "otp": "123456" }
```

Success response (200, login case)

```
{ "success": true, "message": "Login successful", "data": null, "meta": {} }
```

(Sets-Cookie: accessToken=...; HttpOnly, refreshToken=...; HttpOnly)

Error response (400 — wrong code)

```
{ "success": false, "message": "Invalid OTP", "data": null, "meta": {} }
```

POST /api/v1/auth/login

Password login. Rate-limited (`authRateLimit`).

Request body

```
{ "email": "ava.stone@acme.com", "password": "P@ssw0rd123" }
```

Success response (200)

```
{ "success": true, "message": "Login successful", "data": null, "meta": {} }
```

Error response (401)

```
{ "success": false, "message": "Unauthorized", "data": null, "meta": {} }
```

POST /api/v1/auth/refresh-token

Reissues `accessToken` / `refreshToken` from the `refreshToken` cookie. No body.

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": null, "meta": {} }
```

Error response (401)

```
{ "success": false, "message": "Unauthorized", "data": null, "meta": {} }
```

POST /api/v1/auth/logout

Revokes the session tied to the `refreshToken` cookie and clears both cookies. No body.

Success response (200)

```
{ "success": true, "message": "Logged out successfully", "data": null, "meta": {} }
```

POST /api/v1/auth/reset-password

Sets a new password after a forgot-password OTP has been verified. Rate-limited (`authRateLimit`).

Request body

```
{ "channel": "EMAIL", "contact": "ava.stone@acme.com", "newPassword": "N3wP@ssword!" }
```

Success response (200)

```
{ "success": true, "message": "Updated successfully", "data": null, "meta": {} }
```

Error response (404)

```
{ "success": false, "message": "User does not exist", "data": null, "meta": {} }
```


GET /api/v1/auth/social-login

Starts an OAuth flow (currently Salesforce only); returns the authorization URL to redirect the browser to.

Query params

Name	Required	Description		
authProvider	yes	salesforce		
environment	no	production \	sandbox \	custom
customUrl	no	Custom Salesforce domain, required when environment=custom		
userId	no	Pins the OAuth state to an existing user (used for reconnect flows)		

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "authorizationUrl": "https://login.salesforce.com/services/oauth2/authorize?client_id=...&state=8f2c..." },
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "authProvider is required", "data": null, "meta": {} }
```

GET /api/v1/auth/social-login/callback

OAuth redirect target. Exchanges code for tokens, creates/updates the user (and, for the admin-provisioning flow, the org/role/CRM records), sets session cookies.

Query params

Name	Required	Description
authProvider	yes	salesforce
code	yes	OAuth authorization code
state	yes	OAuth state issued by /social-login

Success response (200)

```
{ "success": true, "message": "Login successful", "data": null, "meta": {} }
```

Error response (400 — expired code)

```
{ "success": false, "message": "Salesforce authorization code has expired", "data": null, "meta": {} }
```

POST /api/v1/auth/configure-org

Registers a Salesforce org (and optionally authorizes an admin user) from the managed package's Post-Install/Authorize-Admin flow. Body is an AES envelope encrypted with the shared bootstrap key, not plain JSON.

Request body

```
{
  "cipherText": "5f8a1c9e2b7d4a3f...base64...",
  "iv": "a1b2c3d4e5f60718293a4b5c6d7e8f90",
  "authTag": "0f1e2d3c4b5a69788796a5b4c3d2e1f0"
}
```

Decrypted plaintext shape (for reference):

```
{
  "type": "org",
  "org_details": { "orgId": "00D5g000000XyZAAA0", "instanceUrl": "https://acme.my.salesforce.com", "environment": "production" }
}
```



Success response (200) — payload is itself an encrypted envelope:

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": "{\"cipherText\":\"...\", \"iv\":\"...\", \"authTag\":\"...\"}",
  "meta": {}
}
```

Error response (401 — bad envelope)

```
{ "success": false, "message": "Unauthorized", "data": null, "meta": {} }
```

User (/api/v1/user) — private (authenticate + aclGateway ; user module is in the always-allowed list)

GET /api/v1/user/my-profile

Returns the authenticated user's profile with resolved role permissions; strips password .

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": {
    "userId": "3f6a1e2c-9b4d-4a7e-8c1f-2d5e6a7b8c9d",
    "firstName": "Ava",
    "lastName": "Stone",
    "contact": { "email": "ava.stone@acme.com", "isEmailVerified": true },
    "role": { "name": "Custom", "roleId": "b7e2...", "permissions": ["backup.read", "backup.write"] },
    "isCrmConnected": true,
    "status": "ACTIVE"
  },
  "meta": {}
}
```

Error response (401)

```
{ "success": false, "message": "Unauthorized", "data": null, "meta": {} }
```

PUT /api/v1/user/my-profile

Updates the authenticated user's editable profile fields.

Request body

```
{ "firstName": "Ava", "lastName": "Stone-Reyes", "gender": "female" }
```

Success response (200)

```
{
  "success": true,
  "message": "Updated successfully",
  "data": { "userId": "3f6a1e2c-9b4d-4a7e-8c1f-2d5e6a7b8c9d", "firstName": "Ava", "lastName": "Stone-Reyes", "gender": "female" },
  "meta": {}
}
```

Error response (400 — empty body)

```
{ "success": false, "message": "\"value\" must have at least 1 key", "data": null, "meta": {} }
```

DELETE /api/v1/user/my-profile

Soft-deletes (marks status: DELETED) the authenticated user's account. No body.

Success response (200)

```
{ "success": true, "message": "Deleted successfully", "data": null, "meta": {} }
```

GET /api/v1/user/default-permissions

Returns the static list of default permission keys available to assign to roles.

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": ["backup.read", "backup.write", "backup.delete", "backup.execute", "user.read", "user.write"],
  "meta": {}
}
```

GET /api/v1/user/list

Lists users, optionally paginated/searched.

Query params

Name	Required	Description
pagination	no	"true" to enable cursor pagination
limit	no	Page size, default 10
cursor	no	Opaque pagination cursor
search	no	Free-text search

Success response (200, paginated)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    { "userId": "3f6a1e2c-9b4d-4a7e-8c1f-2d5e6a7b8c9d", "firstName": "Ava", "lastName": "Stone", "contact": { "email": "ava." },
  ],
  "meta": { "limit": 10, "nextCursor": "eyJ1c2VySWQiOiIz...", "totalRecords": 42, "totalPages": 5 }
}
```

GET /api/v1/user/logout

Alternate logout route (revokes the current session by `sessionId` from the access token). No body.

Success response (200)

```
{ "success": true, "message": "Logged out successfully", "data": null, "meta": {} }
```

Error response (401)

```
{ "success": false, "message": "Unauthorized", "data": null, "meta": {} }
```

POST /api/v1/user/change-password

Changes the authenticated user's password.

Request body

```
{ "oldPassword": "P@ssw0rd123", "newPassword": "N3wP@ssword!" }
```

Success response (200)

```
{ "success": true, "message": "Updated successfully", "data": null, "meta": {} }
```

Error response (400 — wrong old password)

```
{ "success": false, "message": "Old password does not match", "data": null, "meta": {} }
```

CRM (/api/v1/crm) — private (always-allowed module, no extra permission check)

GET /api/v1/crm/list

Lists every user/CRM connection sharing the authenticated user's contact email (i.e. every org this person has connected).

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    { "userId": "3f6a1e2c-...", "crmId": "c1a2b3c4-...", "isCrmConnected": true, "crm": { "crmId": "c1a2b3c4-...", "crmName": "salesforce" } },
    ...
  ],
  "meta": {}
}
```

PUT /api/v1/crm/

Renames/updates a connected CRM record.

Request body

```
{ "crmId": "c1a2b3c4-5d6e-7f8a-9b0c-1d2e3f4a5b6c", "name": "Acme Production (EU)" }
```

Success response (200)

```
{
  "success": true,
  "message": "Updated successfully",
  "data": { "crmId": "c1a2b3c4-5d6e-7f8a-9b0c-1d2e3f4a5b6c", "name": "Acme Production (EU)", "crmName": "salesforce" },
  "meta": {}
}
```

Error response (404)

```
{ "success": false, "message": "Does not exist", "data": null, "meta": {} }
```

GET /api/v1/crm/connect

Starts the OAuth flow to (re)connect an existing user's CRM.

Query params

Name	Required	Description		
crmName	no*	salesforce		
userId	no*	Required if crmName is omitted (resolves CRM from the user record)		
environment	no	production \	sandbox \	custom
name	no	Display name to store with the OAuth state		

* at least one of crmName / userId is required.

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": { "redirectUrl": "https://login.salesforce.com/services/oauth2
```

Error response (400)

```
{ "success": false, "message": "id is required", "data": null, "meta": {} }
```

GET /api/v1/crm/callback

OAuth callback for the reconnect flow; stores new tokens against the user.

Query params: `code` (required), `state` (required).

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": null, "meta": {} }
```

Error response (409 — org already registered elsewhere)

```
{ "success": false, "message": "CRM not found", "data": null, "meta": {} }
```

DELETE /api/v1/crm/disconnect

Marks the CRM connection as disconnected for a given user (keeps the CRM/user records, just flips `isCrmConnected`).

Query params: `userId` (required).

Success response (200)

```
{ "success": true, "message": "Updated successfully", "data": null, "meta": {} }
```

Error response (400)

```
{ "success": false, "message": "id is required", "data": null, "meta": {} }
```

DELETE /api/v1/crm/

Deletes a CRM record outright (blocked if backup configs still reference it).

Query params: `crmId` (required).

Success response (200)

```
{ "success": true, "message": "Deleted successfully", "data": null, "meta": {} }
```

Error response (400 — configs still exist)

```
{ "success": false, "message": "Backup configs exist for this CRM", "data": null, "meta": {} }
```

GET /api/v1/crm/refresh-token

Refreshes the Salesforce OAuth token for the connected CRM.

Query params: `crmId` (required).

Success response (200)

```
{
  "success": true,
  "message": "Updated successfully",
  "data": { "access_token": "00D5g00000XyZA!AQ...", "instance_url": "https://acme.my.salesforce.com", "token_type": "Bearer" },
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "crmId is required", "data": null, "meta": {} }
```

CRM Metadata (/api/v1/crm-metadata) — private (always-allowed module)

GET /api/v1/crm-metadata/object-schema

Reads the stored (S3) field schema JSON captured for a backed-up object.

Query params

Name	Required	Description
backupConfigId	yes	Backup config the object belongs to
objetName	yes	Salesforce object API name (note: param is spelled <code>objetName</code> , not <code>objectName</code>)

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "fields": [{ "name": "Id", "type": "id" }, { "name": "Name", "type": "string" }] },
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "Does not exist", "data": null, "meta": {} }
```

GET /api/v1/crm-metadata/objects/list

Lists Salesforce objects available to back up/archive, with record counts, sorted by count descending.

Query params

Name	Required	Description	
crmId	no	Look up objects for a different connected CRM under the same contact email	
mode	no	Legacy alias for <code>type</code>	
type	no	backup \	archival (schedule/realtime split)

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    { "name": "Account", "label": "Account", "count": 15234 },
    { "name": "Contact", "label": "Contact", "count": 8021 }
  ],
  "meta": {}
}
```

GET /api/v1/crm-metadata/objects/describe

Describes a single Salesforce object: candidate child relationships and lookup/master-detail fields.

Query params

Name	Required	Description	
objectName	yes	Salesforce object API name	
mode / type	no	backup \	archival

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": {
    "children": [{ "name": "Contact", "cascadeDelete": true, "restrictedDelete": false, "field": "AccountId" }],
    "fields": [{ "label": "Owner", "referenceTo": ["User"], "name": "OwnerId", "nillable": false, "cascadeDelete": false }]
  },
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "Object name is required", "data": null, "meta": {} }
```

POST /api/v1/crm-metadata/objects/master/list

Given a list of object names, returns the subset that have no lookup/master-detail relationship to another object (i.e. "master" objects with no parent dependency).

Request body

```
{ "objectNames": ["Account", "Contact", "Opportunity"] }
```

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [{ "name": "Account", "label": "Account", "fields": [] }],
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "\"objectNames\" must contain at least 1 items", "data": null, "meta": {} }
```

GET /api/v1/crm-metadata/fields/list

Lists the fields on a Salesforce object for the field-selection UI.

Query params

Name	Required	Description	
objectName	yes	Salesforce object API name	
crmId	no	Look up fields for a different connected CRM	
mode	no	backup \	archival

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [{ "name": "Name", "label": "Account Name", "type": "string" }, { "name": "Industry", "label": "Industry", "type": "string" }, { "name": "Address", "label": "Address", "type": "string" }, { "name": "Phone", "label": "Phone", "type": "string" }, { "name": "Email", "label": "Email", "type": "string" }, { "name": "Website", "label": "Website", "type": "string" }, { "name": "Social Media", "label": "Social Media", "type": "string" }, { "name": "Other", "label": "Other", "type": "string" } ],
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "Unauthorized", "data": null, "meta": {} }
```

Backup Config (/api/v1/backup-config) — private, module backup , permissions

backup.read / backup.write / backup.delete

GET /api/v1/backup-config/objects

Lists Salesforce objects for a CRM annotated with whether they're already backed up. **Permission:** backup.read .

Query params: crmId (required), type / mode (optional).

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "objects": [{ "label": "Account", "apiName": "Account", "isBackedUp": true, "schedule": "schedule" }] },
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "crmId is required", "data": null, "meta": {} }
```

GET /api/v1/backup-config/object-childs

Lists master-detail/cascade-delete child objects for a given object. **Permission:** backup.read .

Query params: crmId (required), objectName , type / mode , relationshipDepth .

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": [{ "name": "Contact", "cascadeDelete": true }], "meta": {} }
```

POST /api/v1/backup-config/objects-count

Returns record counts for a batch of objects. **Permission:** backup.read .

Request body

```
{ "crmId": "c1a2b3c4-5d6e-7f8a-9b0c-1d2e3f4a5b6c", "items": [{ "apiName": "Account" }, { "apiName": "Contact" }] }
```

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "success": true, "results": [{ "success": true, "recordCount": 15234, "apiName": "Account" }, { "success": true,
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "Object name is required", "data": null, "meta": {} }
```

GET /api/v1/backup-config/fields

Lists fields for an object scoped to a backup config. **Permission:** `backup.read` .

Query params: `crmId` (required), `objectName` (required).

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": [{ "name": "Name", "type": "string" }], "meta": {} }
```

POST /api/v1/backup-config/

Creates a backup configuration (schedule- or realtime-based). **Permission:** `backup.write` .

Request body

```
{
  "crmId": "c1a2b3c4-5d6e-7f8a-9b0c-1d2e3f4a5b6c",
  "destinationId": "d9e8f7a6-1234-4b5c-8d9e-0f1a2b3c4d5e",
  "name": "Nightly Account Backup",
  "description": "Full nightly backup of Account + Contact",
  "objectNames": ["Account", "Contact"],
  "schedule": "SCHEDULE",
  "scheduleConfig": {
    "type": "INCREMENTAL",
    "timeZone": "Asia/Kolkata",
    "scheduling": { "frequency": "DAILY", "interval": 1, "startTime": "02:00" }
  },
  "dataset": "STANDARD",
  "status": "ACTIVE"
}
```

Success response (201)

```
{
  "success": true,
  "message": "Created successfully",
  "data": {
    "backupConfigId": "9c8b7a6f-5e4d-3c2b-1a09-8f7e6d5c4b3a",
    "userId": "3f6a1e2c-9b4d-4a7e-8c1f-2d5e6a7b8c9d",
    "crmId": "c1a2b3c4-5d6e-7f8a-9b0c-1d2e3f4a5b6c",
    "name": "Nightly Account Backup",
    "schedule": "SCHEDULE",
    "status": "ACTIVE",
    "backupStatus": "IDLE"
  },
  "meta": {}
}
```

Error response (400 — destination not owned by caller)

```
{ "success": false, "message": "Does not exist", "data": null, "meta": {} }
```

GET /api/v1/backup-config/list

Lists backup configs for the caller (optionally paginated/filtered). **Permission:** `backup.read` .

Query params

Name	Required	Description			
pagination	no	"true" for cursor pagination			
limit , cursor	no	Page size / cursor			
search	no	Free text			
status	no	ACTIVE \	DRAFT \	PAUSED	
backupStatus	no	IDLE \	PENDING \	SUCCESS \	FAILED
schedule	no	SCHEDULE \	REALTIME		

Success response (200, paginated)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    {
      "backupConfigId": "9c8b7a6f-5e4d-3c2b-1a09-8f7e6d5c4b3a",
      "name": "Nightly Account Backup",
      "schedule": "SCHEDULE",
      "backupStatus": "SUCCESS",
      "crm": { "name": "Acme Production", "crmName": "salesforce", "username": "integration@acme.com" },
      "destination": { "name": "Primary S3", "type": "S3" }
    }
  ],
  "meta": { "limit": 10, "nextCursor": null, "totalRecords": 1, "totalPages": 1 }
}
```

GET /api/v1/backup-config/

Fetches one backup config by slug, with resolved CRM/destination details. **Permission:** backup.read .

Query params: slug (required).

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": {
    "backupConfigId": "9c8b7a6f-5e4d-3c2b-1a09-8f7e6d5c4b3a",
    "name": "Nightly Account Backup",
    "slug": "nightly-account-backup",
    "crmDetail": { "crmId": "c1a2b3c4-...", "crmName": "salesforce", "name": "Acme Production", "slug": "acme-production", "
    "destinationDetail": { "destinationId": "d9e8f7a6-...", "destinationName": "Primary S3", "type": "S3" }
  },
  "meta": {}
}
```

Error response (400)

```
{ "success": false, "message": "Backup config not found", "data": null, "meta": {} }
```

PUT /api/v1/backup-config/

Updates a backup config the caller owns. **Permission:** backup.write .

Query params: backupConfigId (required).

Request body

```
{ "name": "Nightly Account Backup (updated)", "status": "PAUSED" }
```

Success response (200)

```
{ "success": true, "message": "Updated successfully", "data": { "backupConfigId": "9c8b7a6f-5e4d-3c2b-1a09-8f7e6d5c4b3a", "n
```

Error response (400 — not owner)

```
{ "success": false, "message": "Does not exist", "data": null, "meta": {} }
```

DELETE /api/v1/backup-config/

Deletes a backup config (and its jobs) the caller owns; blocked while a backup is pending. **Permission:** `backup.delete` .

Query params: `backupConfigId` (required).

Success response (200)

```
{ "success": true, "message": "Deleted successfully", "data": null, "meta": {} }
```

Error response (400 — pending backup)

```
{ "success": false, "message": "Cannot delete while a backup is pending", "data": null, "meta": {} }
```

GET /api/v1/backup-config/stats

Returns job success/failure counters for one backup config (`slug`) or for the whole account. **Permission:** `backup.read` .

Query params: `slug` (optional — omit for account-wide stats).

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": { "total": 42, "success": 40, "failed": 2, "pending": 0 }, "me
```

GET /api/v1/backup-config/italize-payload-transform

Kicks off (fire-and-forget) the EMR payload-transform job for a backup config. **Permission:** `backup.read` .

Query params: `slug` (required).

Success response (201)

```
{ "success": true, "message": "Created successfully", "data": null, "meta": {} }
```

GET /api/v1/backup-config/sync-schema-metadata

Alias route sharing the same handler as `syncMetadataTriggerHandler` per the route table (`sync-metadata` → `syncMetadataTriggerHandler` , `sync-schema-metadata` → `syncMetadataHandler`). Synchronously (re)syns Salesforce object metadata (fields/childs/picklist/recordTypes) for every object in the config. No permission entry currently registered for this specific path in `backupConfigPermissions` (falls through to default-deny unless added).

Query params: `slug` (required).

Success response (200)

```
{ "success": true, "message": "Updated successfully", "data": null, "meta": {} }
```

GET /api/v1/backup-config/sync-metadata

Fire-and-forget trigger for metadata sync (`syncMetadataTriggerHandler`). **Permission:** `backup.read` .

Query params: `slug` (required).

Success response (201)

```
{ "success": true, "message": "Created successfully", "data": null, "meta": {} }
```

Backup Job (/api/v1/backup-job) — private, module backup

GET /api/v1/backup-job/list

Lists backup jobs, either for one backup config (slug) or for the whole account. **Permission:** backup.read .

Query params

Name	Required	Description		
slug	no	Scope to one backup config		
limit , cursor	no	Pagination		
status	no	PENDING \	SUCCESS \	FAILED
startDate , endDate	no	ISO date range (only honored when slug is set)		

Success response (200)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    { "backupJobId": "1e2d3c4b-5a69-4788-9796-a5b4c3d2e1f0", "backupConfigId": "9c8b7a6f-...", "status": "SUCCESS", "destina
  ],
  "meta": { "limit": 10, "nextCursor": null, "totalRecords": 1, "totalPages": 1 }
}
```

GET /api/v1/backup-job/

Fetches a single backup job the caller owns. **Permission:** backup.read .

Query params: backupJobId (required).

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": { "backupJobId": "1e2d3c4b-5a69-4788-9796-a5b4c3d2e1f0", "stat
```

Error response (400)

```
{ "success": false, "message": "Does not exist", "data": null, "meta": {} }
```

GET /api/v1/backup-job/resume

Resumes a stalled/paused backup job. **Permission:** backup.execute .

Query params: backupJobId (required).

Success response (200)

```
{ "success": true, "message": "Resumed successfully", "data": null, "meta": {} }
```

Error response (400)

```
{ "success": false, "message": "id is required", "data": null, "meta": {} }
```

Internal (/api/v1/internal) — server-to-server only, guarded by `internalAuth` (header `X-Internal-Secret` , timing-safe compare), not by `authenticate` / `aclGateway` . Called by backup-service.

GET /api/v1/internal/fields

Proxies field metadata for a backup config (used by backup-service instead of duplicating Salesforce auth).

Headers: `X-Internal-Secret`: <shared secret>

Query params: `backupConfigId` (required), `objectName` (required), `mode` (optional, `backup` / `archival`).

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": [{ "name": "Name", "type": "string" }], "meta": {} }
```

Error response (401 — bad/missing secret)

```
{ "success": false, "message": "Unauthorized", "data": null, "meta": {} }
```

GET /api/v1/internal/picklist-values

Proxies picklist values for one field.

Query params: `backupConfigId` , `objectApiName` , `fieldApiName` (all required).

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": [{ "value": "Hot", "label": "Hot" }, { "value": "Warm", "label": "Warm" }], "meta": {} }
```

GET /api/v1/internal/record-types

Proxies record-type metadata for an object.

Query params: `backupConfigId` , `objectApiName` (required).

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": [{ "recordTypeId": "012...", "name": "Enterprise Account" }], "meta": {} }
```

GET /api/v1/internal/refresh-token

Refreshes the Salesforce token for the CRM tied to a backup config.

Query params: `backupConfigId` (required).

Success response (200)

```
{ "success": true, "message": "Updated successfully", "data": { "access_token": "00D5g000000XyZA!AQ...", "instance_url": "https://example.com" } }
```

POST /api/v1/internal/backup-payload

Webhook backup-service calls to report job/event progress back to client-service (updates `backupStatus` , sizes, schema-change flags; also resumes deferred payload transforms). Always responds 200 immediately, processes async.

Request body

```
{ "eventType": "backup.completed", "eventId": "evt_8f2c1a9b", "backupConfigId": "9c8b7a6f-5e4d-3c2b-1a09-8f7e6d5c4b3a" }
```

Other `eventType` values: `backup.failed`, `backup.size.updated` (adds `objectName`, `sizeInBytes`), `schema.sync.completed`, `schema.updated` (adds `objectName`, `schemaChange`).

Success response (200)

```
{ "success": true, "message": "Updated successfully", "data": null, "meta": {} }
```

Public (/api/v1/public) — no auth middleware (trust boundary is the encrypted payload / decrypt step itself)

POST /api/v1/public/payload

Triggered by an S3/EventBridge-adjacent caller holding the shared transport key; decrypts an envelope to get `backupConfigId` and kicks off the EMR payload-transform job.

Request body

```
{ "payload": "U2FsdGVkX1+8f2c1a9b4d7e...base64-encrypted-string..." }
```

Success response (200)

```
{ "success": true, "message": "Created successfully", "data": null, "meta": {} }
```

Error response (400 — bad payload)

```
{ "success": false, "message": "Invalid payload", "data": null, "meta": {} }
```

POST /api/v1/public/backup-trigger

EventBridge scheduled-rule target; triggers a scheduled backup/archival run for a config.

Request body (EventBridge envelope)

```
{
  "detail": { "backupConfigId": "9c8b7a6f-5e4d-3c2b-1a09-8f7e6d5c4b3a", "userId": "3f6a1e2c-9b4d-4a7e-8c1f-2d5e6a7b8c9d" }
}
```

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": null, "meta": {} }
```

Error response (400)

```
{ "success": false, "message": "Does not exist", "data": null, "meta": {} }
```

PUT /api/v1/public/webhook/salesforce

Salesforce real-time change-event webhook (Data Vault managed package Apex trigger calls this). Body arrives as an encrypted two-layer Salesforce envelope; `attachDecryptedSalesforceRequest('body')` middleware decrypts it and attaches `req.salesforcePayload` — that decryption is the sole authorization check. Responds 200 immediately (fire-and-forget to Salesforce), then fans out one internal call per matching real-time backup config to backup-service.

Request body

```
{
  "cipherText": "b4c3d2e1f0...base64...",
  "iv": "1a2b3c4d5e6f70819223",
  "authTag": "3d4e5f60718293a4b5c6"
}
```

Decrypted plaintext shape (for reference):

```
{
  "transactionId": "txn_7f8e9d0c1b2a",
  "objectApiName": "Account",
  "operation": "update",
  "orgId": "00D5g000000XyZAAA0",
  "records": [{ "Id": "001..." }]
}
```

Success response (200)

```
{ "success": true, "message": "Fetched successfully", "data": null, "meta": {} }
```

Error response (401 — decrypt failed)

```
{ "success": false, "message": "Unauthorized", "data": null, "meta": {} }
```

Endpoint count (Part 1)

Group	Endpoints
Auth	9
User	7
CRM	7
CRM Metadata	5
Backup Config	13
Backup Job	3
Internal	5
Public	3
Total	52

Archival Config

Base path: `/api/v1/archival-config` — all routes private (`authenticate` + `aclGateway` , permission group `archival-config`).

GET `/api/v1/archival-config/object-childs`

Fetches the child/related objects (and their fields) for a Salesforce object via an Apex callout, for archival config building.

Auth: Private — `authenticate` + `aclGateway` (`archival-config` read).

Query params

Name	Type	Required	Description
crmId	string	Yes	Connected CRM id
objectName	string	Yes	Salesforce object API name
type / mode	string	No	Schedule mode (<code>archival \</code> <code>realtime</code>); <code>mode</code> is the legacy alias for <code>type</code>
relationshipDepth	number	No	How many levels of child relationships to traverse

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    {
      "childSObject": "Contact",
      "field": "AccountId",
      "relationshipName": "Contacts",
      "cascadeDelete": true
    }
  ],
  "meta": {}
}
```

Error response 400

```
{
  "success": false,
  "message": "crmId is required",
  "data": null,
  "meta": {}
}
```

POST `/api/v1/archival-config/object-records`

Fetches sample records for an object (with optional parent-chain / condition-derived WHERE clause) via Apex.

Auth: Private.

Request body

```
{
  "crmId": "8f14e45f-ceed-467e-b2f5-cfb7be0f4b1e",
  "apiName": "Contact",
  "fields": ["Id", "Name", "Email"],
  "referenceName": "AccountId",
  "parent": {
    "apiName": "Account",
    "referenceName": "Id",
    "filters": { "condition": { "type": "ALL" }, "fields": null }
  }
}
```

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    { "Id": "003gL000004xyzAAA", "Name": "Jane Doe", "Email": "jane.doe@example.com" }
  ],
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "apiName is required", "data": null, "meta": {} }
```

GET /api/v1/archival-config/fields

Lists field metadata for a Salesforce object (Apex callout).

Query params

Name	Type	Required	Description
crmId	string	Yes	Connected CRM id
objectName	string	Yes	Salesforce object API name

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    { "name": "Email", "label": "Email", "dataType": "EMAIL", "required": false, "filterable": true }
  ],
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "objectName is required", "data": null, "meta": {} }
```

GET /api/v1/archival-config/get-picklist-field-values

Fetches picklist values for a field via Apex.

Query params

Name	Type	Required	Description
crmlId	string	Yes	Connected CRM id
objectApiName	string	Yes	Object API name
fieldApiName	string	Yes	Picklist field API name

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [ { "label": "Hot", "value": "Hot" }, { "label": "Warm", "value": "Warm" } ],
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "crmId, objectApiName and fieldApiName are required", "data": null, "meta": {} }
```

GET /api/v1/archival-config/list

Lists archival configs for the caller, optionally paginated with filters.

Query params

Name	Type	Required	Description	
pagination	"true"	"false"	No	Enables cursor pagination + filters below
limit	number	No	Page size (default 10)	
cursor	string	No	Opaque pagination cursor	
search	string	No	Name search	
status	string	No	Config status filter	
backupStatus	string	No	Backup status filter	
name	string	No	Exact name filter (non-paginated mode)	

Success response 200 (paginated)

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    {
      "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234",
      "name": "Contact Archival",
      "type": "ARCHIVAL",
      "status": "ACTIVE",
      "crm": { "name": "Acme Corp", "crmName": "salesforce" },
      "archivedRecordsCount": 15320,
      "archivedSizeInBytes": 5242880
    }
  ],
  "meta": { "limit": 10, "nextCursor": "eyJpZCI6ImIxZTZjOWQwIn0=", "totalRecords": 42, "totalPages": 5 }
}
```

GET /api/v1/archival-config/

Fetches a single archival config by slug, enriched with CRM and destination summaries.

Query params

Name	Type	Required	Description
slug	string	Yes	Archival config slug

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": {
    "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234",
    "name": "Contact Archival",
    "slug": "contact-archival",
    "type": "ARCHIVAL",
    "crmDetail": { "crmId": "8f14e45f-ceea-467e-b2f5-cfb7be0f4b1e", "crmName": "salesforce", "name": "Acme Corp", "slug": "a",
    "destinationDetail": { "destinationId": "d4a1e2b3-1111-4222-8333-abcdef012345", "destinationName": "Primary S3", "type":
  },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "Backup config not found", "data": null, "meta": {} }
```

GET /api/v1/archival-config/stats

Aggregated archival job stats, scoped to one config by slug or to the caller's user/space.

Query params

Name	Type	Required	Description
slug	string	No	Scope stats to one archival config

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "totalRecords": 15320, "totalSize": 5242880 },
  "meta": {}
}
```

PUT /api/v1/archival-config/

Updates an archival config (objects, schedule, etc.); triggers an immediate backup job for any newly-added immediate objects.

Query params

Name	Type	Required	Description
backupConfigId	string	Yes	Config to update

Request body

```
{
  "name": "Contact Archival v2",
  "objects": [
    {
      "id": "obj-1",
      "name": "Contact",
      "type": "PARENT",
      "field": [ { "name": "Email", "dataType": "EMAIL", "filter": { "operator": "EQUALS", "value": "test@example.com" } } ]
    }
  ],
  "scheduleConfig": { "type": "IMMEDIATE", "timeZone": "America/New_York" }
}
```

Success response 200

```
{
  "success": true,
  "message": "Updated successfully",
  "data": { "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234", "name": "Contact Archival v2", "status": "ACTIVE" },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "backupConfigId is required", "data": null, "meta": {} }
```

DELETE /api/v1/archival-config/

Deletes an archival config (and its backup jobs); blocked while a backup is pending.

Query params

Name	Type	Required	Description
backupConfigId	string	Yes	Config to delete

Success response 200

```
{ "success": true, "message": "Deleted successfully", "data": null, "meta": {} }
```

Error response 400

```
{ "success": false, "message": "Backup is pending, cannot delete", "data": null, "meta": {} }
```

POST /api/v1/archival-config/dry-run

Dry-runs an archival config's SOQL for the given CRM/object tree without persisting anything.

Auth: Private + `dryRunArchivalValidation` (Joi).

Request body

```
{
  "crmId": "8f14e45f-ceea-467e-b2f5-cfb7be0f4b1e",
  "objects": [
    {
      "id": "obj-1",
      "name": "Contact",
      "type": "PARENT",
      "condition": { "type": "SOQL", "soqlQuery": "Email != null" },
      "field": [ { "name": "Email", "dataType": "EMAIL", "filter": { "operator": "EQUALS", "value": "test@example.com" } } ]
    }
  ]
}
```

Success response 201

```
{
  "success": true,
  "message": "Created successfully",
  "data": { "query": "SELECT Id, Email FROM Contact WHERE Email != null", "recordCount": 128 },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "\"crmId\" is required", "data": null, "meta": {} }
```

POST /api/v1/archival-config/validate-soql

Validates a single object's SOQL condition against Salesforce metadata.

Auth: Private + `validateSoqlArchivalValidation` (Joi).

Request body

```
{
  "crmId": "8f14e45f-ceea-467e-b2f5-cfb7be0f4b1e",
  "object": {
    "name": "Contact",
    "condition": { "type": "SOQL", "soqlQuery": "Email != null" },
    "field": [ { "name": "Email", "dataType": "EMAIL", "filter": { "operator": "EQUALS", "value": "test@example.com" } } ]
  },
  "isParent": true
}
```

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "valid": true, "normalizedQuery": "Email != null" },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "\"object.name\" is required", "data": null, "meta": {} }
```

POST /api/v1/archival-config/

Creates a new archival config. If not `DRAFT`, immediately triggers a backup job for any immediate (non-scheduled) objects.

Auth: Private + `createArchivalConfigValidation` (Joi).

Request body

```
{
  "crmId": "8f14e45f-ceea-467e-b2f5-cfb7be0f4b1e",
  "destinationId": "d4a1e2b3-1111-4222-8333-abcdef012345",
  "name": "Contact Archival",
  "description": "Archive stale contacts",
  "objectNames": ["Contact"],
  "schedule": "SCHEDULE",
  "scheduleConfig": {
    "type": "RECURRING",
    "timeZone": "America/New_York",
    "scheduling": { "frequency": "WEEKLY", "interval": 1, "weekDays": ["MON"], "startTime": "02:00" }
  },
  "objects": [
    {
      "id": "obj-1",
      "name": "Contact",
      "type": "PARENT",
      "field": [ { "name": "Email", "dataType": "EMAIL", "filter": { "operator": "EQUALS", "value": "test@example.com" } } ]
    }
  ],
  "status": "ACTIVE"
}
```

Success response 201

```
{
  "success": true,
  "message": "Created successfully",
  "data": { "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234", "name": "Contact Archival", "status": "ACTIVE", "type": "PARENT" },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "\"destinationId\" is required", "data": null, "meta": {} }
```

GET /api/v1/archival-config/record-errors

Returns one page (10 rows) of per-record delete errors for an archival job's object, read from CSV batch files on S3.

Query params

Name	Type	Required	Description
backupJobId	string	Yes	Archival job id
objectId	string	Yes	Object node id within the job's object tree
page	number	No	Page number (default 1)

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": {
    "records": [ { "recordId": "003gL000004xyzAAA", "error": "REQUIRED_FIELD_MISSING: Email" } ],
    "totalRecords": 137,
    "totalPages": 14,
    "page": 1
  },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "backupJobId and objectId are required", "data": null, "meta": {} }
```

Dashboard

Base path: `/api/v1/dashboard` — private.

GET `/api/v1/dashboard/overview`

Returns headline dashboard metrics: protected records, storage used, backup success rate, active jobs — scoped to the user's space (or user, if no space).

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": {
    "protectedRecords": { "value": "1.2M", "change": "+3.4%", "period": "vs last week" },
    "storageUsed": { "value": "482.3 GB", "change": "+1.1%", "period": "vs last week", "progress": 24 },
    "backupSuccessRate": { "value": "99.2%", "period": "Last 7 Day", "status": "healthy" },
    "activeJobs": { "value": 18, "running": 2, "period": "Running" }
  },
  "meta": {}
}
```

GET `/api/v1/dashboard/last-jobs`

Returns the caller's last 10 backup jobs, each annotated with its parent backup config's name and schedule.

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    {
      "backupJobId": "j-9a8b7c6d",
      "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234",
      "status": "COMPLETED",
      "startedAt": "2026-08-20T02:00:00.000Z",
      "backupConfig": { "name": "Contact Backup", "schedule": "SCHEDULE" }
    }
  ],
  "meta": {}
}
```

Destination

Base path: `/api/v1/destination` — private.

POST `/api/v1/destination/`

Creates a storage destination (e.g. an S3 bucket). For AWS/S3 destinations, either validates the supplied credentials or verifies the caller already granted the DataVault Athena role bucket access.

Auth: Private + `createDestinationValidation` (Joi).

Request body

```
{
  "name": "Primary S3",
  "provider": "AWS",
  "type": "S3",
  "config": {
    "bucketName": "acme-datavault-backups",
    "region": "us-east-1",
    "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
    "secretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
    "folderPath": "backups/"
  },
  "is_already_granted": false
}
```

Success response 201

```
{
  "success": true,
  "message": "Created successfully",
  "data": {
    "destinationId": "d4a1e2b3-1111-4222-8333-abcdef012345",
    "name": "Primary S3",
    "provider": "AWS",
    "type": "S3",
    "userId": "u-1234"
  },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "invalid_aws_credentials", "data": null, "meta": {} }
```

GET `/api/v1/destination/list`

Lists the caller's (or their space's) destinations with cursor pagination; decrypts each to surface `bucketName / region`, strips ciphertext.

Query params

Name	Type	Required	Description
limit	number	No	Page size, max 100 (default 20)
cursor	string	No	Opaque pagination cursor

Success response 200


```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    { "destinationId": "d4a1e2b3-1111-4222-8333-abcdef012345", "name": "Primary S3", "type": "S3", "bucketName": "acme-datavault-backups" },
  ],
  "meta": { "nextCursor": "eyJpZCI6ImQ0YTFlMmIzIn0=" }
}
```

GET /api/v1/destination/

Fetches a single destination by id (ownership enforced).

Query params

Name	Type	Required	Description
destinationId	string	Yes	Destination id

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "destinationId": "d4a1e2b3-1111-4222-8333-abcdef012345", "name": "Primary S3", "type": "S3" },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "destinationId is required", "data": null, "meta": {} }
```

GET /api/v1/destination/config

Fetches a destination's decrypted config, with secret keys stripped.

Query params

Name	Type	Required	Description
destinationId	string	Yes	Destination id

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "bucketName": "acme-datavault-backups", "region": "us-east-1", "folderPath": "backups/" },
  "meta": {}
}
```

PUT /api/v1/destination/

Updates a destination.

Auth: Private + `updateDestinationValidation` (Joi).

Query params

Name	Type	Required	Description
destinationId	string	Yes	Destination id

Request body

```
{ "name": "Primary S3 (renamed)" }
```

Success response 200

```
{
  "success": true,
  "message": "Updated successfully",
  "data": { "destinationId": "d4a1e2b3-1111-4222-8333-abcdef012345", "name": "Primary S3 (renamed)" },
  "meta": {}
}
```

DELETE /api/v1/destination/

Deletes a destination (ownership enforced).

Query params

Name	Type	Required	Description
destinationId	string	Yes	Destination id

Success response 200

```
{ "success": true, "message": "Deleted successfully", "data": null, "meta": {} }
```

Restore Retrieve

Base path: `/api/v1/restore` — private.

POST `/api/v1/restore/`

Creates a restore request; unless saved as `DRAFT`, immediately creates and kicks off its restore job (EMR/Spark transform).

Auth: Private + `createRestoreValidation` (Joi).

Request body

```
{
  "crmId": "8f14e45f-ceea-467e-b2f5-cfb7be0f4b1e",
  "source": {
    "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234",
    "type": "ENTIRE"
  },
  "selection": {
    "restoreScope": { "type": "ALL" }
  },
  "destination": { "type": "SAME", "tagRestoredRecord": "Restored_By_DataVault__c" },
  "conflict": { "restoreMode": "SKIP" },
  "jobDetail": { "name": "Q3 Contact Restore", "description": "Restore before mass update", "tags": ["q3", "contacts"] },
  "schedule": { "type": "IMMEDIATE", "timeZone": "America/New_York" },
  "status": "PENDING"
}
```

Success response `201`

```
{ "success": true, "message": "Created successfully", "data": null, "meta": {} }
```

Error response `400`

```
{ "success": false, "message": "\"source.backupConfigId\" is required", "data": null, "meta": {} }
```

POST `/api/v1/restore/activate`

Transitions a `DRAFT` restore to `PENDING` and starts its restore job.

Request body

```
{ "restoreId": "r-1234abcd" }
```

Success response `200`

```
{ "success": true, "message": "Updated successfully", "data": null, "meta": {} }
```

Error response `400`

```
{ "success": false, "message": "restore_not_draft", "data": null, "meta": {} }
```

GET `/api/v1/restore/config/list`

Paginated list of restore configs for the caller, with optional search/status/date filters.

Query params

Name	Type	Required	Description
limit	number	No	Page size (default 10)
cursor	string	No	Opaque pagination cursor
search	string	No	Name search
status	string	No	Status filter
createdAtFrom	string (ISO date)	No	Range start
createdAtTo	string (ISO date)	No	Range end

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [ { "restoreId": "r-1234abcd", "status": "COMPLETED", "createdAt": "2026-08-10T09:00:00.000Z" } ],
  "meta": { "limit": 10, "nextCursor": "eyJpZCI6InItMTIzNCJ9" }
}
```

GET /api/v1/restore/job

Returns the latest restore job for a given restoreId, with encrypted destination tokens and raw source stripped.

Query params

Name	Type	Required	Description
restoreId	string	Yes	Restore id

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "restoreJobId": "rj-9988", "restoreId": "r-1234abcd", "status": "COMPLETED", "destination": { "type": "SAME" } },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "restoreId is required", "data": null, "meta": {} }
```

GET /api/v1/restore/job/stats

Aggregated restore job stats — scoped to a restore, or to the caller if restoreId omitted.

Query params

Name	Type	Required	Description
restoreId	string	No	Scope to one restore

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "totalJobs": 4, "completedJobs": 3, "failedJobs": 0, "processedRecordCount": 5000, "successRecordCount": 4990 },
  "meta": {}
}
```

GET /api/v1/restore/list

Paginated list of restore/retrieve jobs, scoped to a backup config or the whole user.

Query params

Name	Type	Required	Description
backupConfigId	string	No	Scope to one config
limit	number	No	Page size (default 10)
cursor	string	No	Opaque pagination cursor
status	string	No	Status filter

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [ { "backupJobId": "j-9a8b7c6d", "status": "COMPLETED", "destination": { "type": "S3" } } ],
  "meta": { "limit": 10, "nextCursor": "eyJpZCI6ImotOWE4YiJ9", "totalRecords": 25, "totalPages": 3 }
}
```

GET /api/v1/restore/get-objectlist-by-configid

Returns the object list selected on a given backup/archival config.

Query params

Name	Type	Required	Description	
backupConfigId	string	Yes	Config id	
configType	"BACKUP"\	"ARCHIVAL"	Yes	Must match the config's stored type

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [ { "id": "obj-1", "name": "Contact" } ],
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "invalid_config_type", "data": null, "meta": {} }
```

GET /api/v1/restore/fetch-change-between-backup-jobs

Returns the backup job ids of a config whose runs started inside a given time window, newest first.

Query params

Name	Type	Required	Description
backupConfigId	string	Yes	Config id
startTime	string (ISO date)	Yes	Window start
endTime	string (ISO date)	Yes	Window end
limit	number	No	Page size
cursor	string	No	Opaque pagination cursor

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": ["j-9a8b7c6d", "j-1122aabb"],
  "meta": { "limit": 20, "nextCursor": "eyJpZCI6ImotOTk5In0=" }
}
```

Error response 400

```
{ "success": false, "message": "invalid_time_range", "data": null, "meta": {} }
```

POST /api/v1/restore/retrieve/fetch-records

Fetches records for one object from the compressed Hudi/Delta tables — either the entire object or only what changed inside a window — each row tagged with the restore `OPERATION` (`INSERT` / `DELETE` / `UPDATE`).

Request body

```
{
  "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234",
  "objectApiName": "Contact",
  "type": "CHANGED_BETWEEN",
  "startDate": "2026-08-01T00:00:00.000Z",
  "endDate": "2026-08-20T23:59:59.000Z",
  "columnNames": ["Id", "Name", "Email"],
  "searchText": "jane",
  "cursor": null
}
```

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": {
    "rows": [ { "Id": "003gL000004xyzAAA", "Name": "Jane Doe", "Email": "jane.doe@example.com", "OPERATION": "UPDATE" } ]
  },
  "meta": { "limit": 50, "hasMore": true, "nextCursor": "eyJvZmZzZXQiOiUwfkQ==" }
}
```

Error response 400

```
{ "success": false, "message": "column_names_required", "data": null, "meta": {} }
```

POST /api/v1/restore/retrieve/fetch-inactive-record-types

Returns Record Types made inactive or deleted inside a date window, from stored schema-change deltas.

Request body

```
{
  "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234",
  "objectApiName": "Contact",
  "startDate": "2026-08-01T00:00:00.000Z",
  "endDate": "2026-08-20T23:59:59.000Z"
}
```

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [ { "recordTypeId": "012gL0000004ABC", "name": "VIP Contact", "deletedAt": "2026-08-15T10:00:00.000Z" } ],
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "date_range_required", "data": null, "meta": {} }
```

POST /api/v1/restore/retrieve/fetch-missing-fields

Returns fields deleted from an object inside a date window.

Request body

```
{
  "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234",
  "objectApiName": "Contact",
  "startDate": "2026-08-01T00:00:00.000Z",
  "endDate": "2026-08-20T23:59:59.000Z"
}
```

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [ { "fieldApiName": "Legacy_Score__c", "dataType": "NUMBER", "deletedAt": "2026-08-12T00:00:00.000Z" } ],
  "meta": {}
}
```

GET /api/v1/restore/fetch-object-fields

Returns the latest S3-stored schema for an object under a backup config.

Query params

Name	Type	Required	Description
objectApiName	string	Yes	Object API name
backupConfigId	string	Yes	Backup config id

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [ { "name": "Email", "dataType": "EMAIL" }, { "name": "Name", "dataType": "STRING" } ],
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "not_exist", "data": null, "meta": {} }
```

GET /api/v1/restore/get-picklist-field-values

Picklist values for a field, read from the S3-persisted schema (not a live Apex callout).

Query params

Name	Type	Required	Description
backupConfigId	string	Yes	Backup config id
objectApiName	string	Yes	Object API name
fieldApiName	string	Yes	Field API name

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [ { "label": "Hot", "value": "Hot" }, { "label": "Cold", "value": "Cold" } ],
  "meta": {}
}
```

GET /api/v1/restore/restore

Returns a single restore/retrieve job by `backupJobId`, sanitized (encrypted source/destination fields removed).

Query params

Name	Type	Required	Description
backupJobId	string	Yes	Job id

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "backupJobId": "j-9a8b7c6d", "status": "COMPLETED", "destination": { "type": "S3" } },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "not_exist", "data": null, "meta": {} }
```

Salesforce

Base path: `/api/v1/salesforce` — **public** (no `authenticate` / `aclGateway` ; `auth` is instead a per-request encrypted-envelope handshake specific to the connected Salesforce org, via `attachDecryptedSalesforceRequest`). All bodies/queries are an encrypted `envelope` the middleware decrypts into `req.salesforcePayload = { crm, plaintext }` ; responses are re-encrypted with `encryptSalesforceResponse` and are NOT wrapped in the standard `{ success,message,data,meta }` envelope.

GET /api/v1/salesforce/permissions

Returns the static permission-module catalog (unauthenticated, unencrypted — plain JSON).

Success response `200`

```
{
  "modules": [
    {
      "key": "backup",
      "label": "Backup",
      "description": "Manage backup configurations",
      "actions": [
        { "key": "view", "label": "View", "description": "View backups", "risky": false },
        { "key": "delete", "label": "Delete", "description": "Delete backups", "risky": true }
      ]
    }
  ]
}
```

GET /api/v1/salesforce/user/list

Lists DataVault users provisioned for the connected org, with role/permission info. Query carries the encrypted envelope.

Query params

Name	Type	Required	Description
envelope	string (JSON)	Yes	{ ciphertext, iv } encrypted with the org's key

Success response `200` (plaintext shown decrypted for illustration; wire body is { ciphertext, iv })

```
{
  "users": [
    {
      "userId": "005gL000001AbCdQAK",
      "name": "Jane Doe",
      "email": "jane.doe@example.com",
      "modules": { "backup": ["view", "config"] },
      "roleId": "role-123",
      "roleName": "Backup Admin",
      "lastSyncDate": "2026-08-19T10:00:00.000Z"
    }
  ]
}
```

POST /api/v1/salesforce/user-update

Creates, updates, or deletes DataVault users/roles to match the org's Salesforce user/permission state (called from an Apex batch sync).

Auth: `attachDecryptedSalesforceRequest('body')` + `upsertUsersValidation` (Joi, validated against decrypted plaintext).

Request body (decrypted plaintext shape)

```
{
  "organizationId": "00DgL000000abcXYZ",
  "environment": "production",
  "users": [
    {
      "firstName": "Jane",
      "lastName": "Doe",
      "profile": {
        "organizationId": "00DgL000000abcXYZ",
        "instanceUrl": "https://acme.my.salesforce.com",
        "userId": "005gL000001AbCdQAK",
        "username": "jane.doe@acme.com",
        "email": "jane.doe@example.com",
        "photoUrl": ""
      },
      "role": { "permissions": [] }
    }
  ]
}
```

Success response 201

```
[
  { "firstName": "Jane", "lastName": "Doe", "status": "success", "action": "created" }
]
```

Error response 400

```
{ "errorCode": "VALIDATION_FAILED" }
```

GET /api/v1/salesforce/role/list

Lists roles for the connected org's CRM record.

Query params

Name	Type	Required	Description
envelope	string (JSON)	Yes	Encrypted envelope

Success response 200

```
{
  "roles": [
    { "roleId": "role-123", "name": "Backup Admin", "description": "", "modules": { "backup": ["view", "config"] }, "created": "2020-01-01T00:00:00Z" }
  ]
}
```

POST /api/v1/salesforce/role/create

Creates a new role for the connected org.

Auth: `attachDecryptedSalesforceRequest('body')` + `createRoleValidation` (Joi).

Request body (decrypted)

```
{ "orgId": "00DgL000000abcXYZ", "name": "Read Only", "modules": { "backup": ["view"] }, "createdBy": "jane.doe@acme.com" }
```

Success response 201

```
{ "roleId": "role-456" }
```

Error response 400

```
{ "errorCode": "VALIDATION_FAILED" }
```

PUT /api/v1/salesforce/role/update

Updates an existing role's name/permissions.

Auth: attachDecryptedSalesforceRequest('body') + updateRoleValidation (Joi).

Request body (decrypted)

```
{ "orgId": "00DgL000000abcXYZ", "roleId": "role-456", "name": "Read Only (updated)", "modules": { "backup": ["view"], "dashb
```

Success response 200

```
{ "roleId": "role-456", "name": "Read Only (updated)", "permissions": { "backup": ["view"], "dashboard": ["view"] } }
```

Error response 404

```
{ "errorCode": "ROLE_NOT_FOUND" }
```

DELETE /api/v1/salesforce/role/delete

Deletes a role.

Query params

Name	Type	Required	Description
envelope	string (JSON)	Yes	Encrypted envelope containing { roleId }

Success response 200

```
{ "success": true }
```

Error response 404

```
{ "errorCode": "ROLE_NOT_FOUND" }
```

GET /api/v1/salesforce/confirm-admin-user-created

Confirms whether the admin user created during org setup now exists in DataVault.

Query params

Name	Type	Required	Description
envelope	string (JSON)	Yes	Encrypted envelope containing { userId }

Success response 200

```
{ "success": true }
```

Error response 404

```
{ "errorCode": "USER_NOT_FOUND" }
```

GET /api/v1/salesforce/confirm-org-authorized

Bootstrap-only check for whether an org id is already authorized/registered — decrypted with the platform's own `ENCRYPTION_KEY`, not an org-specific key (org may not have one yet). Uses the standard `makeResponse` envelope (unlike the other salesforce endpoints).

Query params

Name	Type	Required	Description
envelope	string (JSON)	Yes	Encrypted envelope containing { <code>orgId</code> }

Success response 200

```
{ "success": true, "message": "Fetched successfully", "data": { "success": true }, "meta": {} }
```

Error response 404

```
{ "success": false, "message": "crm_not_found", "data": null, "meta": {} }
```

Settings

Base path: /api/v1/settings — private.

GET /api/v1/settings/

Fetches user (optionally CRM-scoped) settings.

Query params

Name	Type	Required	Description
crmId	string	No	Scope settings to one CRM

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "userId": "u-1234", "crmId": "8f14e45f-ceeda-467e-b2f5-cfb7be0f4b1e", "standardObjects": ["Contact", "Account"],
  "meta": {}
}
```

PUT /api/v1/settings/

Upserts user settings.

Auth: Private + upsertSettingsValidation (Joi).

Request body

```
{ "crmId": "8f14e45f-ceeda-467e-b2f5-cfb7be0f4b1e", "standardObjects": ["Contact", "Account", "Opportunity"], "status": "ACTIVE" }
```

Success response 200

```
{
  "success": true,
  "message": "Updated successfully",
  "data": { "userId": "u-1234", "crmId": "8f14e45f-ceeda-467e-b2f5-cfb7be0f4b1e", "standardObjects": ["Contact", "Account", "Opportunity"],
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "\"status\" must be one of [ACTIVE, PAUSED]", "data": null, "meta": {} }
```

Spark Job

Base path: `/api/v1/spark-job` — **public** at the router level (no `authenticate` / `ac1Gateway`); secured instead by request-body/envelope encryption verifiable only by services holding the shared encryption key (Spark/EMR jobs, internal callers).

POST `/api/v1/spark-job/build-payload`

Called by the Spark/EMR compression or restore job to fetch its full build payload. Body is an encrypted envelope; response is a raw base64-encoded JSON string (not the standard envelope) because Spark's `JsonUtils.java` expects exactly that.

Request body

```
{ "payload": "eyJjaXB0ZXJ0ZXh0IjoilLi4uIiwiaXYiOiIuLi4ifQ==" }
```

Decrypts to either:

```
{ "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234" }
```

or

```
{ "restoreConfigId": "r-1234abcd" }
```

Success response `200` (raw base64 string body, decoded here for illustration)

```
{
  "details": {
    "objectOperations": {
      "j-9a8b7c6d": { "objectApiName": "Contact", "operation": "COMPRESS" }
    }
  }
}
```

Error response `400`

```
{ "success": false, "message": "invalid_payload", "data": null, "meta": {} }
```

POST `/api/v1/spark-job/update-spark-job-status`

Terminal callback from Spark reporting compression or restore-object success/failure for a config.

Request body

```
{ "payload": "eyJjaXB0ZXJ0ZXh0IjoilLi4uIiwiaXYiOiIuLi4ifQ==" }
```

Decrypts to (BACKUP variant):

```
{
  "type": "BACKUP",
  "backup": { "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234", "backupJobIds": ["j-9a8b7c6d", "j-1122aabb"], "succe
}
```

or (RESTORE variant):

```
{
  "type": "RESTORE",
  "restore": { "restoreConfigId": "rj-9988", "objects": ["Contact", "Account"], "success": true }
}
```

Success response 200

```
{
  "success": true,
  "message": "Updated successfully",
  "data": { "updated": ["j-9a8b7c6d", "j-1122aabb"], "failed": [] },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "backup_config_not_found", "data": null, "meta": {} }
```

GET /api/v1/spark-job/get-inactive-owner-ids

Resolves the org's Salesforce tokens for a CRM id and queries inactive users (`IsActive = false`), optionally also returning distinct manager ids.

Query params

Name	Type	Required	Description	
crmId	string	Yes	Connected CRM id	
includeManagers	"true"\	"false"	No	Also return distinct <code>ManagerId</code> s

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": { "ownerIds": ["005gL000001AbCdQAK"], "managerIds": ["005gL000001XyzEfQAK"] },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "crm_not_connected", "data": null, "meta": {} }
```

Storage

Base path: /api/v1/storage — private.

GET /api/v1/storage/overview

Returns storage size records, the last backup job, and monthly stats for a CRM (from `crm-id` header or the user's default CRM).

Headers

Name	Required	Description
crm-id	No	Overrides the user's default CRM

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": {
    "backupConfigSizeRecord": { "totalSizeBytes": 5242880000, "recordCount": 1200000 },
    "lastBackupJob": { "backupJobId": "j-9a8b7c6d", "status": "COMPLETED", "startedAt": "2026-08-20T02:00:00.000Z" },
    "monthlyStats": [ { "month": "2026-08", "sizeBytes": 5242880000, "recordCount": 1200000 } ]
  },
  "meta": {}
}
```

Error response 400

```
{ "success": false, "message": "crm_id_required", "data": null, "meta": {} }
```

GET /api/v1/storage/last-backup-config

Returns the last 5 backup configs of the given type for a CRM.

Headers

Name	Required	Description
crm-id	No	Overrides the user's default CRM

Query params

Name	Type	Required	Description
type	"NORMAL"\\	"ARCHIVAL"	No Config type filter (default <code>NORMAL</code>)

Success response 200

```
{
  "success": true,
  "message": "Fetched successfully",
  "data": [
    { "backupConfigId": "b1e6c9d0-8b3a-4c2a-9f3a-2f7a0e0e1234", "name": "Contact Backup", "sizeBytes": 1048576 }
  ],
  "meta": {}
}
```

Error response 400


```
{ "success": false, "message": "crm_id_required", "data": null, "meta": {} }
```